

EPIC 2 User Story 2 : Tenant Middleware &

Story Metadata

- **Story ID:** 1.2
- **Priority:** High
- **Estimate:** 5 SP

User Statement

As a Security Engineer,

I want a robust, split-layer pipeline for tenant resolution (Extraction vs. Validation),

So that I can ensure zero-leakage security and comprehensive observability without blocking the event loop or duplicating logic in endpoints.

Acceptance Criteria (Definition of Done)

1. Extraction Layer (Middleware)

- **Identification:** Extracts Tenant ID from `X-Tenant-ID` header with a fallback to Subdomain via `X-Forwarded-Host` or `Host`.
- **Context Initialization:** Sets `tenant_context` (`ContextVar`) immediately for logging before any database or validation logic occurs.
- **Non-Blocking:** The middleware layer does **not** call the DB or raise errors; it simply passes `tenant=unknown` if resolution fails.

2. Validation Layer (Gatekeeper Dependency)

- **Centralized Enforcement:** Use a single dependency (e.g., `get_current_tenant_id`) to validate the tenant. No manual resolution in endpoint code.
- **Allowlist Bypass:** Public endpoints (`/health`, `/docs`, `/openapi.json`, `/auth/*`, `/tenants/resolve`) skip validation.
- **Strict Rejection:** All protected endpoints return `403 Forbidden` if the tenant is missing, invalid, or does not exist in the DB.
- **Token Cross-Check:** If a JWT is present, the `tenant_id` in the token must match the resolved header; mismatch results in a `403()`

3. Performance & Observability

- **Caching:** Implements `async-lru` to cache DB tenant lookups (TTL: 5 mins) to prevent database thrashing.
 - **Logging:** Every request log entry must include `tenant_id` (or `unknown`), ensuring traceability even for 404s or 500s.
 - **DB Filtering:** The resolved `tenant_id` is attached to the request state and used as the primary key for downstream DB query filtering.
-

Execution Point's Considerations

1. Resolution Logic & JWT

- **Order:** Domain name (Web App) → Mobile Header (`X-Tenant-ID` or `X-Tenant-Slug`).
- **Proxy Support:** Use `X-Forwarded-Host` as the primary source (Coolify compatibility).
- **JWT Integrity:** JWTs must be strictly tied to the tenant to prevent cross-tenant access.

2. Public Endpoint Allowlist

Maintain a **single centralized list** to avoid hardcoding across the codebase:

- `/health`, `/docs`, `/openapi.json`
- `/auth/*`
- `/tenants/resolve` (Subdomain → Tenant mapping)
- `/config/public` (Themes/Logos)

3. Middleware Enforcement (Production)

- **Logic Flow:** Resolve tenant → If missing on protected path, return 403 → Validate existence efficiently → Attach to `request.state.tenant_id` and `request.state.tenant_slug`.
- **Performance:** Middleware must use **per-instance caching** to avoid heavy DB work on every request.

4. Data Access & Filtering

- **Dependency Helper:** Use `get_tenant_id(request)` to ensure consistent filtering and easy test coverage.
- **Enforcement Strategy:** * *Option A:* Explicitly require `tenant_id` in every repository/service method.
 - *Option B (Production Gold Standard):* Use **PostgreSQL Row-Level Security (RLS)** to block cross-tenant reads/writes at the DB level.

5. Security Hardening

- **Input Validation:** Accept only UUIDs for `tenant_id` or slugs matching

`^[a-z0-9-]{3,50}$`.

- **Throttling:** Implement rate-limiting for invalid tenant requests at the API Gateway level.
- **Enumeration:** Prefer 403 for missing/invalid tenants to reduce the risk of tenant enumeration.

6. Logging & Deployment (Coolify/VPS)

- **Structured Logging:** Include `request_id`, `tenant_id/slug`, `user_id`, `path`, `method`, `status_code`, and `latency_ms`.
 - **Tracing:** Use or generate an `X-Request-ID` header.
 - **Reliability:** Use multiple workers (Uvicorn/Gunicorn) with per-worker caching. Ensure TLS/HSTS and correct CORS configuration.
-

Technical Tasks

- [] **Setup:** Add `async-lru` to dependencies.
 - [] **Middleware:** Implement `core/middleware.py` (Header extraction + `ContextVar`).
 - [] **Dependency:** Create `get_current_tenant_id` in `api/deps.py` (Validation + Caching).
 - [] **Logging:** Update `logging_config.py` to pull from `ContextVar`.
 - [] **Testing:** * Verify public bypass.
 - Verify 403 on missing/invalid headers.
 - Test JWT tenant mismatch.
 - Confirm cache hits for repeat lookups.
-

Task 1.2.1: Infrastructure & Dependencies

Objective: Prepare the Python environment to support high-performance, non-blocking caching.

Standard Python caching (`functools.lru_cache`) is synchronous and will block the entire server during database lookups. We must install an asynchronous alternative.

- **Action Items:**
 - Open `requirements.txt`.
 - Add the line: `async-lru==2.0.4`.
 - Run `pip install -r requirements.txt` in your terminal.
- **Definition of Done:**
 - [] The library is installed without error.
 - [] You can run `import async_lru` in a Python shell without an `ImportError`.

@requirements.txt

Act as a Senior Python DevOps Engineer. We are implementing Infrastructure Setup for Non-Blocking Caching.

****Objective:**** Add the ``async-lru`` library to our project dependencies to support asynchronous caching for database lookups. Standard ``functools.lru_cache`` is blocking and unsuitable for our FastAPI architecture.

****Action Items:****

1. Check ``requirements.txt`` to see if ``async-lru`` is already present.
2. If not, append exactly: ``async-lru==2.0.4`` to the file.
3. Do NOT modify or remove any existing dependencies.
4. After editing the file, provide the terminal command to install the new requirements.
5. Create a one-line verification command I can run to prove the import works.

****Constraints:****

- Use exact version ``2.0.4``.
- Maintain the existing file structure.

Completed.

Above lib will saved in docker container not in my mac

Task 1.2.2: Context Infrastructure (The "Invisible Backpack")

Objective: Create a thread-safe mechanism to store the `tenant_id` for the duration of a request.

This is critical because in `async` Python, requests are handled concurrently on a single thread. A global variable would be overwritten constantly. We use `ContextVar` to ensure data isolation.

- File: `backend/core/tenant_context.py` (New File)
- Technical Implementation:
 - Initialize a `ContextVar` named `tenant_context` with a default value of `"unknown"`.
 - Create a helper function `set_tenant_id(tenant_id: str)` to write to the context.
 - Create a helper function `get_tenant_id() -> str` to read from the context.
- Definition of Done:
 - [] The file exists.
 - [] The `tenant_context` variable is not exposed directly; only the helper functions are used.

Cursor AI Prompt

@backend/core/

Act as a Senior Python Backend Engineer specializing in Asyncio and FastAPI.

****Task:**** Create the Context Infrastructure file for our Multi-Tenant SaaS.

****File Path:**** `backend/core/tenant\_context.py` (Create new file)

****Objective:****

Implement a thread-safe, async-safe mechanism to store the `tenant\_id` for the duration of a single request.

We are using `contextvars` to ensure data isolation between concurrent requests on the same event loop.

****Technical Specifications:****

1. ****Initialize ContextVar:**** Create a `ContextVar` named `tenant\_context` with a default value of `"unknown"`.
2. ****Encapsulation:**** Do NOT export the `tenant\_context` variable directly. It should be private/internal.
3. ****Set Helper:**** Create a function `set\_tenant\_id(tenant\_id: str) -> None` that sets the value.
4. ****Get Helper:**** Create a function `get\_tenant\_id() -> str` that returns the current value.
5. ****Documentation:**** Add a module-level docstring explaining that this file handles request-scoped context storage.

****Constraints:****

- Use standard library `contextvars`.
- Ensure strict Type Hinting.
- The default value must be explicit (`"unknown"`).

****Definition of Done:****

- The file allows setting a tenant ID in middleware and retrieving it in deep service layers without passing arguments.

Test Plan :-

Checked the code manually

Task 1.2.3: Observability & Logging

Objective: Ensure that every single log line in the application automatically includes the `tenant_id`.

This allows us to debug issues like "Why did School A get a 500 error?" by filtering logs for `tenant_id="school_a"`, even if the error happened deep in a utility function.

- File: `backend/core/config/logging_config.py`
 - Technical Implementation:
 - Import `get_tenant_id` from `core.tenant_context`.
 - Locate the `JSONFormatter` (or your custom formatter class).
 - Inside the `format` method, inject a new key-value pair: `"tenant_id": get_tenant_id()`.
 - Definition of Done:
 - [] Run the app locally.
 - [] Logs output `{"level": "INFO", "tenant_id": "unknown", "message": "..."} by default.`
 - [] (Test) Manually call `set_tenant_id("test")` in `main.py` startup, and verify logs show `tenant_id="test"`.
-

Task 1.2.4: Tenant Extraction Middleware

Objective: Build the "Extraction Layer" that identifies *who* is calling the API.

This layer is non-blocking. It does not check the database. Its only job is to look at the request headers and put a label on the request.

- File: `backend/core/middleware.py` (New File)
- Technical Implementation:
 - Create a class `TenantContextMiddleware` inheriting from `BaseHTTPMiddleware`.
 - Step 1 (Header): Check for `X-Tenant-ID`. If present, use it.
 - Step 2 (Subdomain - Fallback): If header is missing, check `X-Forwarded-Host` (for production) or `Host` (for local). Extract the subdomain (e.g., `math` from `math.tutorapp.com`).
 - Step 3 (Set Context): Call `set_tenant_id(resolved_id)` immediately.
 - Step 4 (Store State): Set `request.state.tenant_id = resolved_id`.
- Definition of Done:

- [] Middleware successfully extracts **X-Tenant-ID**.
 - [] Middleware successfully extracts **math** from **math.localhost**.
 - [] Middleware correctly handles IP addresses (ignores them or sets to **None**).
 - [] No database queries are triggered here.
-

headaTask 1.2.5: Application Wiring

Objective: Activate the middleware globally so it runs for every incoming request.

- File: **backend/main.py**
 - Technical Implementation:
 - Import **TenantContextMiddleware**.
 - Add it to the application stack:
app.add_middleware(TenantContextMiddleware).
 - Ensure it is added *after* **CORSMiddleware** (if applicable) but *before* any other logic middleware.
 - Definition of Done:
 - [] Application starts up successfully.
 - [] Hitting **/health** returns 200 OK (Public endpoints must not break).
-

Task 1.2.6: The Gatekeeper Dependency (Validation)

Objective: Build the "Enforcement Layer" that validates the tenant against the database.

This is where security happens. If the tenant ID is fake, deleted, or missing, we reject the request here.

- File: **backend/api/deps.py**
- Technical Implementation:
 - Import **async_lru**.
 - Cache Helper: Create **_check_tenant_exists(id, session)** decorated with **@alru_cache(ttl=300)**. It performs a lightweight **SELECT 1 FROM tenants WHERE id = ?**.
 - Dependency: Create **get_current_tenant_id(request: Request, session: AsyncSession)**.
 - Allowlist Logic: If **request.url.path** is in **[/health, /docs, ...]**, return **None**

immediately.

- Validation Logic:
 1. Read `request.state.tenant_id`.
 2. If `None` -> Raise `HTTPException(403, "Tenant Required")`.
 3. Call `await _check_tenant_exists(...)`.
 4. If `False` -> Raise `HTTPException(403, "Invalid Tenant")`.
 - Return: The valid `tenant_id`.
 - Definition of Done:
 - [] Public endpoints are accessible without headers.
 - [] Protected endpoints return 403 if header is missing.
 - [] Protected endpoints return 403 if `X-Tenant-ID: fake_school`.
 - [] Protected endpoints return 200 if `X-Tenant-ID: school_a`.
 - [] Database logs show queries only happen once every 5 minutes (due to caching).
-

Task 1.2.7: Route Protection

Objective: Secure the API endpoints by enforcing the new dependency.

- File: `backend/api/v1/endpoints/*.py` (All route files)
- Technical Implementation:
 - Update all router functions to include the dependency:
`tenant_id: str = Depends(deps.get_current_tenant_id)`
 - Pass this `tenant_id` to your CRUD service/controllers.
- Definition of Done:
 - [] You cannot access *any* business data (Books, Users) without a valid Tenant ID.
 - [] Golden Test: A request with `X-Tenant-ID: school_a` successfully retrieves data for School A.